CE 222 – *Computer Organization and Assembly Language (COAL)*
Lecture 16

Dr. Asad Mahmood

Feb. 23, 2026

Lecture Outline

- Announcements ()
- Outline of Previous Lecture:
  - **Quiz 1 Solution**
  - **Chapter 3 – Arithmetic for Computers**
    - 3.1 Introduction
    - 3.2 Addition and Subtraction

- Outline to Today's Lecture:
  - **Chapter 3 – Arithmetic for Computers**
    - 3.3 Multiplication

Chapter 3

Arithmetic for Computers

Chapter Theme

Numerical precision is the very soul of science.

Sir D'arcy Wentworth Thompson, *On Growth and Form, 1917*

Introduction

Numerical precision is the very soul of science.

Sir D'arcy Wentworth Thompson,

On Growth and Form, 1917

Chapter Outline

Chapter 3 - Arithmetic for Computers

- 3.1 Introduction
- 3.2 Addition and Subtraction
- 3.3 Multiplication
- 3.4 Division
- 3.5 Floating Point
- 3.6 Parallelism and Computer Arithmetic: Subword Parallelism
- 3.7 Real Stuff: Streaming SIMD Extensions in x86
- 3.8 Going Faster: Subword Parallelism and Matrix Multiply
- 3.9 Fallacies and Pitfalls
- 3.10 Concluding Remarks
- 3.11 Historical Perspective and Further Reading

Introduction

- Computer operates on integers using their binary representation
- Arithmetic operations are very important for a processor
- Common arithmetic operations on integers
  - Addition and subtraction
  - Multiplication and division
  - Dealing with overflow
- Dealing with real numbers
- Implications of operations implementations on ISA

Chapter Outline

Chapter 3 - Arithmetic for Computers

- 3.1 Introduction
- 3.2 Addition and Subtraction
- 3.3 Multiplication
- 3.4 Division
- 3.5 Floating Point
- 3.6 Parallelism and Computer Arithmetic: Subword Parallelism
- 3.7 Real Stuff: Streaming SIMD Extensions in x86
- 3.8 Going Faster: Subword Parallelism and Matrix Multiply
- 3.9 Fallacies and Pitfalls
- 3.10 Concluding Remarks
- 3.11 Historical Perspective and Further Reading

Integer Addition

- As studied in logic design
- Example: 7 + 6
- Overflow
  - When adding +ve and –ve operands
  - When adding two +ve or two –ive operands
  - How to detect?

Integer Subtraction

- Add negation of second operand
- Example: 7 – 6

- Overflow
  - Subtracting two +ve or two –ve operands
  - Subtracting +ve from –ve operand
  - Subtracting –ve from +ve operand

Arithmetic Operations

- With RISC-V 32-bit registers, we will need an extra bit to detect overflows
- Overflow in unsigned nos.
- Some languages like C may not give correct answer when overflow happens -> Solution?
- Implementations of these arithmetic operations greatly impact processor performance

Arithmetic for Multimedia

- Graphics and media processing operates on vectors of 8-bit and 16-bit data
  - Use 64-bit adder, with partitioned carry chain
    - Operate on 8×8-bit, 4×16-bit, or 2×32-bit vectors
  - SIMD (single-instruction, multiple-data)
- Saturating operations
  - On overflow, result is largest representable value
    - c.f. 2s-complement modulo arithmetic
  - E.g., clipping in audio, saturation in video

Chapter Outline

Chapter 3 - Arithmetic for Computers

- 3.1 Introduction
- 3.2 Addition and Subtraction
- 3.3 Multiplication
- 3.4 Division
- 3.5 Floating Point
- 3.6 Parallelism and Computer Arithmetic: Subword Parallelism
- 3.7 Real Stuff: Streaming SIMD Extensions in x86
- 3.8 Going Faster: Subword Parallelism and Matrix Multiply
- 3.9 Fallacies and Pitfalls
- 3.10 Concluding Remarks
- 3.11 Historical Perspective and Further Reading

Multiplication

"Multiplication is vexation, Division is as bad; The rule of three doth puzzle me, And practice drives me mad.

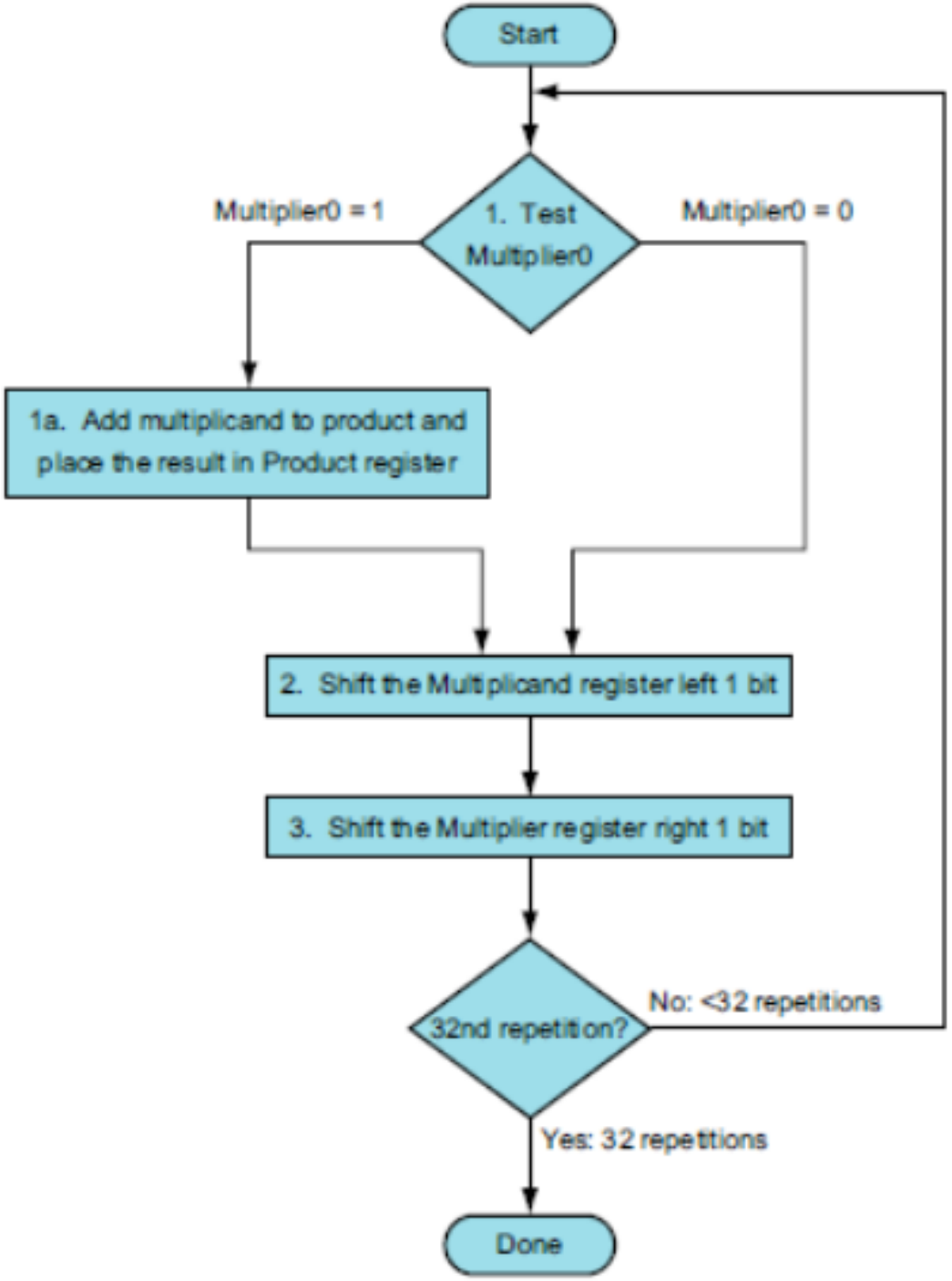
Anonymous, Elizabethan manuscript, 1570

Multiplication

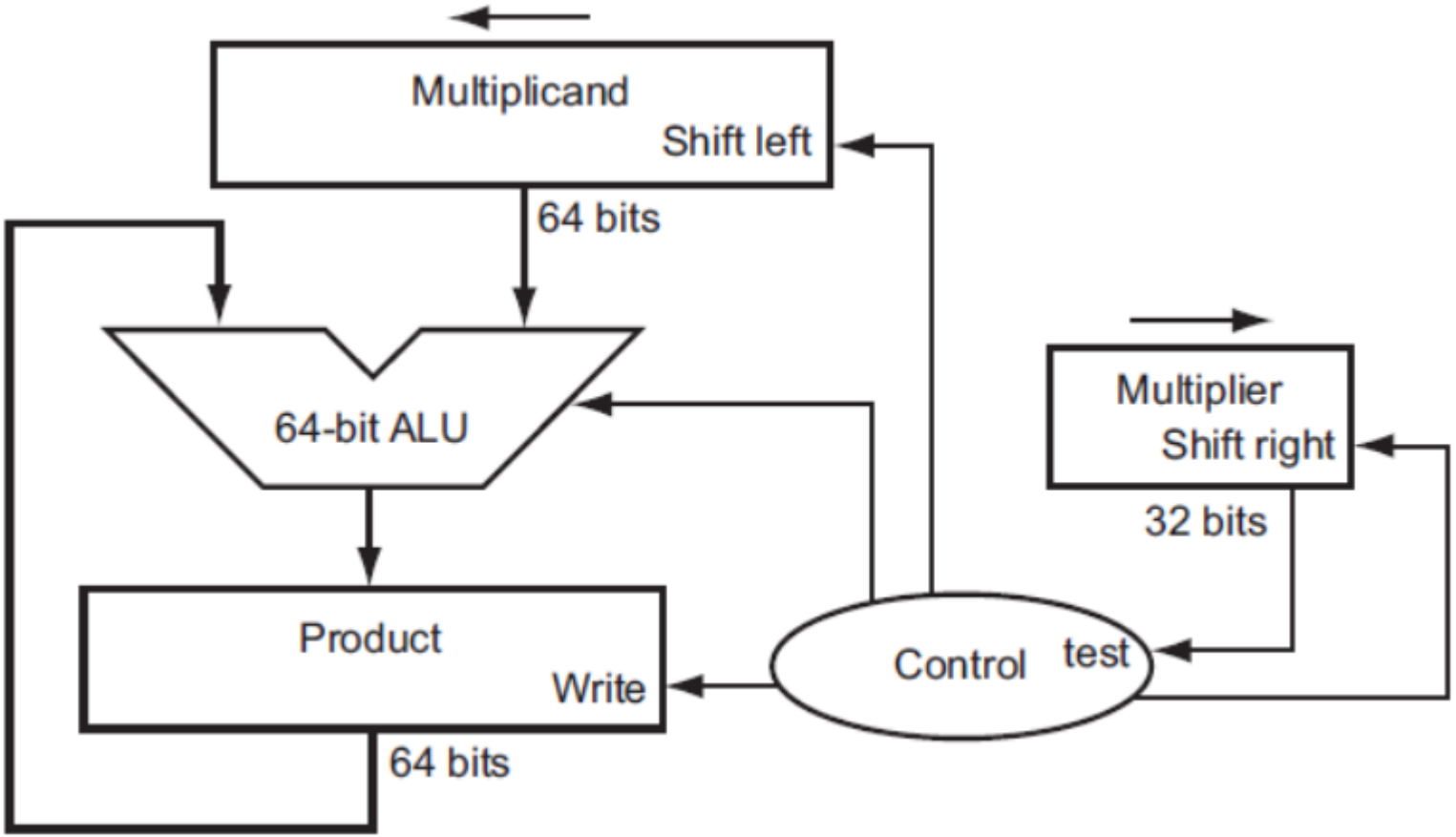
- How do we multiply binary numbers?
- Length of product is the sum of operand lengths
  - Implications for RISC-V?

- We will go through evolution of binary multipliers instead of only presenting the most optimized/recent version

Multiplication Algorithm



Multiplication

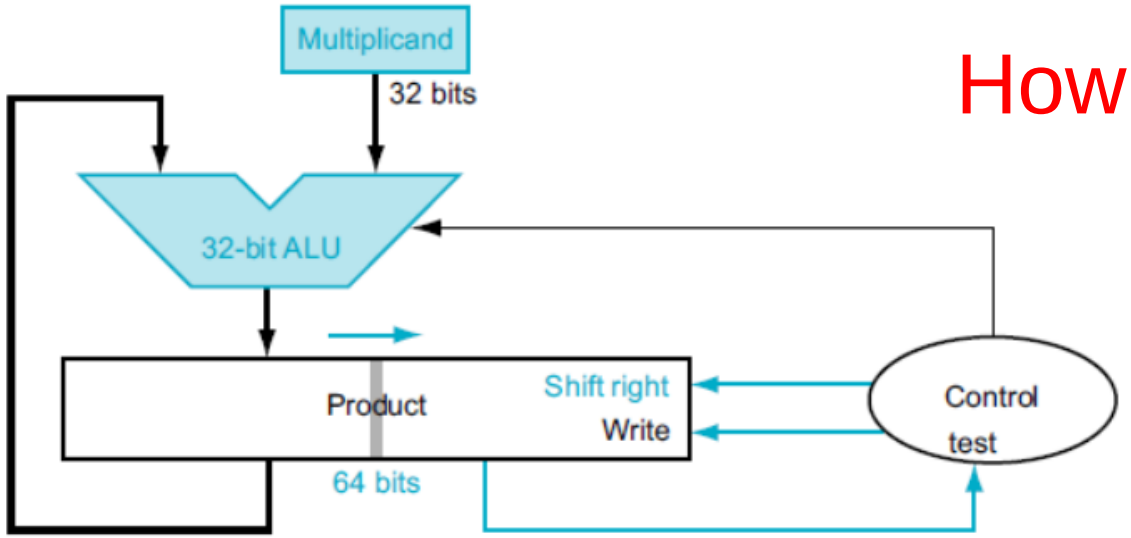


Multiplication Algorithm Example

Iteration	Step	Multiplier	Multiplicand	Product
0	Initial values	0011	0000 0010	0000 0000
1	1a: 1 ⇒ Prod = Prod + Mcand	0011	0000 0010	0000 0010
	2: Shift left Multiplicand	0011	0000 0100	0000 0010
	3: Shift right Multiplier	0001	0000 0100	0000 0010
2	1a: 1 ⇒ Prod = Prod + Mcand	0001	0000 0100	0000 0110
	2: Shift left Multiplicand	0001	0000 1000	0000 0110
	3: Shift right Multiplier	0000	0000 1000	0000 0110
3	1: 0 ⇒ No operation	0000	0000 1000	0000 0110
	2: Shift left Multiplicand	0000	0001 0000	0000 0110
	3: Shift right Multiplier	0000	0001 0000	0000 0110
4	1: 0 ⇒ No operation	0000	0001 0000	0000 0110
	2: Shift left Multiplicand	0000	0010 0000	0000 0110
	3: Shift right Multiplier	0000	0010 0000	0000 0110

Optimized Multiplier

- Perform steps in parallel



How optimized?

- One cycle per partial-product addition
  - That's ok, if frequency of multiplications is low

Signed Multiplication

- So far, we have only looked at multiplication of positive numbers
- How to multiply signed numbers?